NexusSaaS

Gesamtspezifikation & Entwickler-Handoff

Zusammenführung aus Konzept/Blueprint und Vollständigkeitsprüfung – ergänzt um eine umsetzbare Laufzeit-, Sicherheits- und Betriebsanleitung.

Version 1.0 · 6. November 2026

1. Zweck und Status

Dieses Dokument konsolidiert die zwei vorliegenden NexusSaaS-PDFs und ergänzt sie um Runtime View, State-Management, Sandbox-Härtung, Multi-Tenant-Isolation, Webhook-Idempotenz, OAuth-Sicherheitsentscheidungen, Backup-/Löschabläufe, NFRs, Teststrategie und Umsetzungsreihenfolge. Das Ergebnis ist eine belastbare technische Spezifikation für ein MVP und einen Senior-Engineer-Handoff – keine Garantie für „Enterprise-ready“ oder rechtliche Konformität ohne Implementierung, Tests, Betriebsnachweise und Datenschutzprüfung.

Wichtige Korrektur: Die ursprüngliche Spezifikation bezeichnet OAuth2 Password Flow als Vorgabe. Für eine neue produktive Anwendung soll dieser Grant nicht verwendet werden; OAuth 2.0 Security BCP sagt ausdrücklich, dass der Resource Owner Password Credentials Grant nicht verwendet werden darf. Für Browser-/Mobile-Clients verwende Authorization Code + PKCE oder einen etablierten OIDC-Provider. [RFC 9700, Abschnitt 2.4]

2. Architekturentscheidungen (ADRs)

ID	Entscheidung	Begründung / Folge
ADR-01	Ein modularer Backend-Service plus separater Worker/Orchestrator im MVP.	Agenten als klar abgegrenzte Graph-Nodes; Microservices erst bei konkretem Skalierungs-/Isolationserfordernis.
ADR-02	LangGraph für zustandsbehaftete Agentenabläufe; PostgreSQL als Checkpointer.	Interrupts und Wiederaufnahme brauchen Checkpointer und stabile thread_id; Redis bleibt Queue/Rate-Limit/Cache. [LangGraph Interrupts]
ADR-03	Große Artefakte in Object Storage, nicht in Graph-State.	State speichert IDs/URIs/Hashes; vermeidet unnötige Checkpoint-Payloads und trennt Artefakt-Lifecycle vom Workflow.
ADR-04	Untrusted Generated Code läuft isoliert in separater Sandbox-Infrastruktur.	MVP mindestens gehärteter rootless Runner; für öffentliche/high-risk Ausführung gVisor oder MicroVM und keine Docker-Socket-Mounts.
ADR-05	PostgreSQL RLS ergänzt Applikationsautorisierung.	Tenant-ID in tenantbezogenen Tabellen, RLS mit Default-Deny; RLS ersetzt keine Tests und privilegierte DB-Rollen können sie umgehen. [PostgreSQL RLS]
ADR-06	Stripe-Events werden mit DB-Unique-Key idempotent verarbeitet.	Ereignis-ID und Zustandsänderung transaktional; externe Nebenwirkungen über Outbox/Worker.
ADR-07	Art. 17 wird nicht mit Soft-Delete gleichgesetzt.	Soft-delete kann Zwischenstatus sein; Löschung/Anonymisierung, gesetzliche Aufbewahrung und Backups sind gesondert zu behandeln.

3. Zielarchitektur

Bausteine

• Web/API: FastAPI, Auth/OIDC integration, REST API, request validation, RBAC, tenant context.

• Orchestrator: LangGraph graph runner; durable workflow state/checkpoints in PostgreSQL; controlled model/tool adapters.

• Worker pool: asynchronous generation, test, scanning, artifact packaging, webhook jobs.

• Data: PostgreSQL (business data, tenant RLS, webhook inbox/outbox), Redis (short-lived cache, rate limit and queue only where suitable), object storage (generated source, logs/artifacts, reports).

• Sandbox service: separate host/account/network boundary; receives immutable artifact bundle; returns structured result and logs; cannot access application secrets.

• External systems: LLM provider(s), Stripe, email provider, object storage, observability. Treat each as a trust boundary with timeouts, quotas, secret isolation, and failure handling.

Datenfluss in Kurzform

Client → API authentication/authorization → create generation job → persist job + outbox event → worker starts LangGraph with tenant-scoped thread_id → blueprint validation → schema/DDL → code generation → deterministic

checks → isolated sandbox test → security gates → human approval → artifact publication. Status/event updates are persisted and streamed to the client. Failed steps are retried only when idempotent, otherwise routed to manual review.

4. Laufzeitsicht und Ablauf

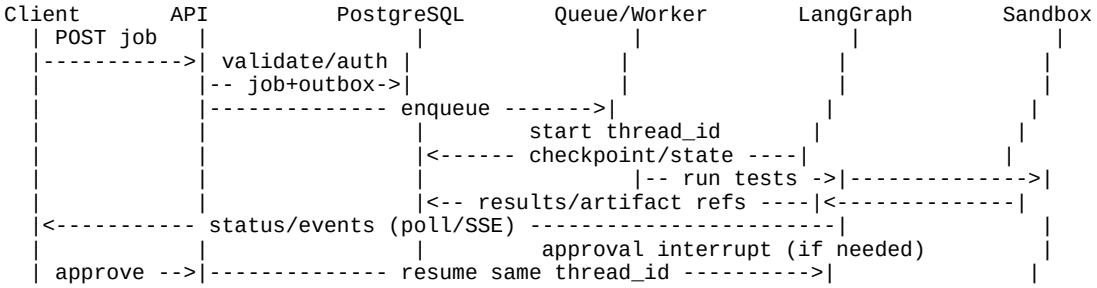
Workflow-Zustände

• RECEIVED → SPEC_VALIDATION → BLUEPRINT_APPROVAL (optional HITL) → DDL_GENERATION → BACKEND_GENERATION → STATIC_CHECKS → SANDBOX_TEST → SECURITY_REVIEW → HUMAN_RELEASE_APPROVAL → PACKAGED → COMPLETED.

• Fehlerpfad: NODE_FAILURE → classify (transient / validation / security / budget) → retry with bounded policy OR REFINE (max 3 repairs) OR PIVOT (explicitly approved strategy change) OR FAILED_REVIEW.

• Jeder Zustandswechsel schreibt job status, event timestamp, tenant_id, trace_id, actor/tool, attempt counter und schema version. Geheime Werte und unnötige personenbezogene Daten gehören nicht in Logs oder den Graph-State.

Sequenzdiagramm (Textform)



LangGraph-Regeln

Für Interrupts einen persistenten Checkpointer und stabile thread_id verwenden. Beim Resume dieselbe ID verwenden; Nodes können nach Resume von vorn starten. Deshalb Seiteneffekte vor Interrupts idempotent halten, nicht-idempotente Aktionen hinter Freigaben legen oder in separate Nodes verschieben. Interrupt-Payloads serialisierbar halten; Resume-Eingaben serverseitig validieren. [LangGraph Interrupts]

5. State, Checkpoints und Artefakte

• State schema versionieren und typisieren: tenant_id, job_id, workflow_version, blueprint_ref, generated_artifact_ref, test_result_ref, security_findings_ref, status, attempt, budget_usage, trace_id.

• Keine vollständigen Repositories, Base64-Dateien, Secrets oder unnötigen Chat-Historien in Checkpoints speichern. Artefakte als Object-Storage-Objekt mit content hash, tenant scope, version, retention, encryption metadata referenzieren.

• thread_id als opake UUID oder serverseitig generierter Identifier; nie vom Client ungeprüft als Tenant-Autorisierung übernehmen. Autorisierung jedes Resume-/Read-/Cancel-Aufrufs unabhängig prüfen.

• Checkpointer-Migration/Setup als kontrollierter Deployment-Schritt betreiben; Start-up-Rennen vermeiden. Bibliotheksversion, Migrationsverhalten und API vor Umsetzung anhand der aktuell gepinnten offiziellen LangGraph-Version verifizieren.

• Retention: abgeschlossene Threads nach definierter fachlicher Frist löschen/anonymisieren; Abhängigkeiten und Restore-Pfade testen. Vorläufige Richtlinie: Job-Logs 30 Tage, Checkpoints 30 Tage nach Abschluss, Artefakte 30 Tage, es sei denn Vertrag/Aufbewahrungspflicht oder konkrete Produktanforderung verlangt anderes. Diese Werte sind Startannahmen, keine Rechtsvorgabe.

6. Multi-Tenancy und Datenbank

Tenant-Modell

• Mandant ist first-class entity (tenants.id); users/tenant_memberships modellieren Mitgliedschaften und Rollen. Tenant-ID aus validiertem Auth-Claim bzw. Membership-Lookup beziehen, niemals allein aus Body/Header vertrauen.

• Jede tenantbezogene Tabelle erhält tenant_id NOT NULL, passende zusammengesetzte Schlüssel/Indizes und Fremdschlüssel, die Tenant-Grenzen berücksichtigen.

• PostgreSQL RLS: ENABLE und FORCE ROW LEVEL SECURITY; Policies für SELECT/INSERT/UPDATE/DELETE mit USING und WITH CHECK; Anwendungsrolle ist kein Tabellenowner und besitzt kein BYPASSRLS. PostgreSQL weist darauf hin, dass Tabellenowner typischerweise RLS umgehen und FORCE dies ändert. [PostgreSQL RLS]

• Setze Tenant-Kontext transaktionslokal (z. B. SET LOCAL) pro DB-Transaktion und teste Connection-Pool-Reuse; bei Fehlern muss der nächste Request nicht den Kontext des vorherigen Requests erben.

• RLS ist Defense-in-Depth, kein Ersatz für API-Autorisierung, tenant-aware object keys, queue scoping, log redaction und isolierte Tests. Backup-Prozesse müssen sicherstellen, dass RLS nicht still Zeilen aus Backups auslässt. [PostgreSQL RLS]

Beispiel-SQL, als Startpunkt

```
ALTER TABLE projects ENABLE ROW LEVEL SECURITY;
ALTER TABLE projects FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON projects
  USING (tenant_id = current_setting('app.tenant_id')::uuid)
  WITH CHECK (tenant_id = current_setting('app.tenant_id')::uuid);
-- Set within each transaction: SET LOCAL app.tenant_id = '<authorized UUID>';
-- Run application queries with a non-owner role without BYPASSRLS.
```

Das SQL ist ein Muster, kein ungeprüfter Copy/Paste-Produktionspatch: Rollen, Migrationen, Connection pooling, Jobs und Admin-Workflows müssen separat geprüft werden.

7. Sandbox-Sicherheit

Grundsatz: generierter Code ist untrusted. Ein Standard-Container allein ist keine starke Sicherheitsgrenze, weil Container den Host-Kernel teilen. Bei öffentlich zugänglicher Codeausführung deshalb gVisor/MicroVM oder ein dedizierter Sandbox-Dienst erwägen und Risiko/Bedrohungsmodell dokumentieren.

• Keine Docker-Socket-Mounts in den Runner oder generierten Container.

• Rootless, non-root UID, read-only root filesystem, drop capabilities, no-new-privileges, seccomp/AppArmor, CPU/RAM/PID/IO limits, harte Laufzeit- und Output-Limits.

• Netzwerk standardmäßig deaktiviert; erforderlicher Egress ausschließlich über allowlistenden Proxy. Keine Cloud-Metadata-Endpunkte, interne Services oder Datenbankzugriffe aus der Sandbox.

• Keine Produktions-Secrets/Cloud-Credentials in Sandbox-Environment; kurzlebige, eng begrenzte Artefakt-Download-Tokens nutzen.

• Pro Lauf frische Sandbox, isoliertes Workspace/Volume, eindeutige Job-ID, unveränderliches Image per Digest; nach Ende Container und temporäre Daten entfernen.

• Vor Ausführung: Parse/compile, Ruff, Bandit/Semgrep oder vergleichbare deterministische Scanner, dependency/license scan, tests. LLM-Audit ist ergänzend, nicht Sicherheits-Gate.

• Output (Logs, Archive) als untrusted behandeln: Größenlimits, Pfadtraversal-Schutz, Malware-Scanning soweit passend, keine automatische Ausführung beim Download.

8. Auth, RBAC und API

Auth-Vorgabe aktualisieren

Das alte Blueprint sieht OAuth2 Password Flow und 60-Minuten-Access-Tokens vor. Für neue Produktiv-Clients ist das zu revidieren: RFC 9700 sagt, der Resource Owner Password Credentials Grant darf nicht verwendet werden; für Browser-/Native-Apps Authorization Code + PKCE nutzen, alternativ OIDC bei einem etablierten Identity Provider. Access-Tokens kurzlebig und eng auf Audience/Scopes begrenzt; Refresh-Token-Rotation und Replay Detection nach

Clienttyp prüfen. [RFC 9700]

API	Zweck / Sicherheitsanforderung
POST /api/v1/jobs	Job starten; auth, tenant membership, idempotency key, quota check.
GET /api/v1/jobs/{id}	Status/Ergebnis nur innerhalb Tenant; object-level authorization.
POST /api/v1/jobs/{id}/resume	Nur autorisierte HITL-Antwort; schema-validiert, audit trail.
POST /api/v1/jobs/{id}/cancel	Abbruch idempotent; Container-/Worker-Abbruch best effort + Status persistieren.
GET /api/v1/users/me/export	Auskunftsworkflow, Auth + re-auth where appropriate, export nur eigene Personendaten.
DELETE /api/v1/users/me	Löschanforderung starten, Status/Exceptions dokumentieren, keine Gleichsetzung mit sofortigem Soft-delete.
POST /api/v1/webhooks/stripe	Signatur prüfen, idempotent inbox, schnelle 2xx nach persistierter Annahme; Verarbeitung asynchron.
GET /healthz	Liveness ohne sensitive internals.
GET /readyz	Readiness inkl. notwendiger dependencies, keine secrets/detail leaks.

9. Webhooks, Queues und Nebenläufigkeit

• Stripe-Webhooks: validate signature on raw body; persist event_id with UNIQUE constraint before acknowledging; process in a transaction with business-state update. Store event type, received_at, status, attempts, last_error category, not full sensitive payload unless necessary.

• For emails, billing side effects and external calls: transactional outbox. A worker claims records safely, deduplicates by idempotency key, retries transient errors with exponential backoff+jitter, dead-letters permanent failures.

• Use optimistic version columns or row-level locks for competing updates; define transaction boundaries and isolation. Do not hold DB transactions open while calling LLMs/Stripe.

• Queue messages carry job_id, tenant_id, schema/version and correlation ID; workers re-authorize/validate tenant context rather than trusting message fields blindly.

10. Rate-Limiting und Kostenkontrolle

• Redis token bucket per tenant + user + endpoint class; distinct limits for interactive API, generation, export and webhook endpoints.

• Atomic Redis operation (Lua or equivalent), TTL cleanup, bounded key cardinality; return 429 with Retry-After and documented quota headers.

• Hard per-job budgets: max model calls, max input/output tokens, max wall time, max repairs (T_max=3), max concurrent jobs per tenant, max sandbox starts.

• LLM routing based on task class and quality evaluation; pin allowed models, track provider/model version and cost estimate; user approval before paid high-cost actions if required by product policy.

• Circuit breaker/timeouts for external providers; retries only for transient errors and with budget-aware limits.

11. DSGVO / Privacy by Design

Dies ist technische Architekturhilfe, keine Rechtsberatung. Art. 17 enthält Voraussetzungen und Ausnahmen; Löschung, gesetzliche Aufbewahrung und Backups müssen mit Datenschutzbeauftragten/Rechtsberatung und konkreten Verarbeitungszwecken abgestimmt werden.

• Dateninventar und Verarbeitungstätigkeiten: pro Feld Zweck, Rechtsgrundlage (fachlich festzulegen), Empfänger, Speicherort, Retention, Zugriff, Export-/Löschpfad. Datensparsamkeit standardmäßig.

• Art. 15: Exportprozess umfasst relevante personenbezogene Daten und gesetzlich geforderte Zusatzinformationen; JSON kann ein Produktformat sein, nicht automatisch die vollständige rechtliche Erfüllung.

• Art. 17: Soft-delete ist nur Status-/Workflowhilfe. Löschpipeline muss aktive DB, Object Storage, Search/Cache, Logs, Checkpoints, Queue, Drittanbieter und Backups betrachten; notwendige Aufbewahrung/Art.-17-Ausnahmen dokumentieren.

• Backups: Verschlüsselt, Zugriff strikt begrenzt, feste Rotation und Löschfristen; Backups sind „beyond use“ zu halten, Restore-Prozess führt Deletion Registry nach, bevor Daten wieder produktiv zugänglich werden. Löschwünsche nicht als universelle sofortige Überschreibung jedes unveränderlichen Backups versprechen.

• Crypto-shredding nur bei sauberem Key-Design, separater Schlüsselhierarchie und geprüftem Backup-Key-Lifecycle als Option bewerten; kein automatischer Ersatz für rechtliche Prüfung.

• PII in Logs/Traces vermeiden oder minimieren; Datenzugriffe auditieren, Audit-Log selbst minimieren und Retention definieren. AES-256-GCM ist Verschlüsselung, Passwörter dagegen mit Argon2id/geeignetem Password Hashing hashen, niemals reversibel verschlüsseln.

• Incident response: Erkennung, Klassifizierung, Verantwortlichkeiten, Fristen, Beweissicherung und Meldungsentscheidung dokumentieren; kein Audit-Log an sich erfüllt automatisch Meldepflichten.

12. Nichtfunktionale Anforderungen (NFRs)

Die folgenden Zielwerte sind vorläufige MVP-Abnahmekriterien, keine gemessenen Produkteigenschaften. Im Lasttest und anhand der Produktanforderungen festlegen und justieren.

Bereich	MVP-Akzeptanzkriterium (Vorschlag)
API-Latenz	P95 < 500 ms für einfache synchron beantwortete API-Routen (ohne LLM/Joblauf), unter vereinbarter Testlast.
Jobsteuerung	Jobannahme bestätigt binnen 2 s P95; länger laufende Generierung asynchron mit Status/Events.
Zuverlässigkeit	Keine doppelten Stripe-Geschäftsaktionen in Duplikat-/Paralleltests; Jobresume nach Worker-Neustart aus persistentem Checkpoint.
Tenant-Sicherheit	Cross-tenant Read/Write/Export/Resume in automatisierten Tests immer verweigert; RLS + API tests.
Sandbox	Keine Netzwerk-Egress-Route ohne explizite Allowlist; harte CPU/RAM/PID/Timeout-/Outputlimits.
Wiederherstellung	RPO/RTO vor Produktion festlegen, Backup restore regelmäßig testen; Deletion Registry beim Restore anwenden.
Kosten	Per-tenant quotas und per-job token/call caps; bei Budgeterschöpfung kontrolliert stoppen und Status melden.
Observability	Jeder Job mit correlation_id, tenant-safe logs, Metriken, Fehlerkategorien; keine Secrets/PII in Standardlogs.
Accessibility/Operations	Runbook für Deploy, rollback, migrations, outage, data export/deletion und incident response.

13. Test- und Freigabestrategie

• Unit: Blueprint schema, state transitions, retry controller, permissions, serialization, idempotency helpers.

• Integration: PostgreSQL migrations, RLS under app-role, tenant session reset with pool reuse, checkpointer save/resume, Redis token bucket, Stripe signature/inbox/outbox.

• Security: tenant boundary tests, object authorization/BOLA, dependency scanning, secret scanning, SAST, sandbox network/filesystem/resource boundary. Keine echten Angriffsausführungsrezepte in Produktionsumgebung; Tests nur in autorisierten isolierten Targets.

• Resilience: worker kill/restart, duplicate queue delivery, provider timeout, DB failover, partial artifact upload, cancellation and retry exhaustion.

• E2E: sample blueprint → generated artifact → static checks → sandbox tests → human approval → package; verify trace and retention deletion.

• Release gate: zero critical/high unresolved scanner findings (oder dokumentierte Ausnahme), passing tests, migration review, rollback tested, privacy/security sign-off and artifact provenance recorded.

14. Umsetzungsplan

Phase	Umfang	Abnahme
0 – Entscheidungen	Use cases, threat model, tenant model, chosen IDP, sandbox service, object store, retention/privacy assumptions.	ADRs akzeptiert; NFRs und data inventory abgestimmt.
1 – Plattformgrundlage	Repo/CI, FastAPI skeleton, PostgreSQL/Alembic, Redis, config/secrets, observability, health/readiness.	Deploybar in dev; migrations/rollback; basic traces.
2 – Identity & tenants	OIDC/Auth, tenant membership/RBAC, tenant_id schema, RLS policies, API authorization.	Cross-tenant tests grün, pool reuse sicher.
3 – Job API + Graph	Jobs, durable queue, typed LangGraph state, persistent checkpointer, status/resume/cancel, bounded retries.	Crash/resume und idempotency tests grün.
4 – Synthesis pipeline	Blueprint validator, DDL generation, code generation, test/scanner nodes, object storage pointers.	Reproducible fixture outputs and artifacts.
5 – Sandbox	Separate runner, hardened profiles, network deny, quotas, disposable workspace, output checks.	Threat-model-based boundary tests.
6 – Billing/operations	Stripe webhook inbox/outbox, plans/quotas, rate limiting, backup/restore, deletion/export processes.	Duplicate/reorder webhook tests; restore and delete drills.
7 – Pilot	Limited tenants, human release approval, cost/quality monitoring, incident runbooks.	Pilot exit criteria achieved before wider launch.

15. Repository-Struktur

```
nexussaas/
  apps/api/                # FastAPI routes, dependencies
  apps/worker/             # queue consumers, scheduled jobs
  packages/domain/          # entities, policies, interfaces
  packages/blueprint/       # JSON Schema + validators + fixtures
  packages/orchestration/   # LangGraph state, nodes, routing
  packages/security/        # auth, tenant context, audit, scanner adapters
  packages/sandbox/         # runner client; no Docker socket in app
  packages/storage/         # PostgreSQL, object storage, Redis
  migrations/               # Alembic + RLS policies
  infra/                    # IaC, deployment, sandbox hosts
  tests/unit/ integration/ security/ e2e/
  docs/adr/ runbooks/ threat-model/
  pyproject.toml            # pinned dependencies + tools
  compose.yaml              # development only
```

16. Review der ursprünglichen Vorgaben

• OAuth2 Password Flow: streichen für neue Produktiv-Clients; OIDC/Authorization Code + PKCE festlegen. [RFC 9700]

• Soft-delete als Art.-17-Erfüllung: Aussage korrigieren; es ist kein automatischer Ersatz für Löschung/Anonymisierung und die Ausnahmen/Retention müssen konkret beurteilt werden.

• Docker --rm: Lebenszyklusoption, keine Sicherheitsgarantie; isolation, network, privileges, limits und separate host boundary festlegen.

• Unveränderliches Audit-Log wegen Art. 33: Art. 33 handelt von Meldung einer Verletzung; Logs sind ein Kontrollmittel, nicht daraus automatisch verpflichtend/unveränderlich. Zweck, Minimierung und Retention definieren.

• „Deterministisch“: Workflow-Kontrollfluss/Validierung kann deterministisch sein, LLM-Ausgaben sind nicht ohne Weiteres deterministisch; Modellversion, Parameter, Prompt/schema version, artifacts und evaluator results mitschreiben.

• „Vollständig Enterprise-ready“ oder „ohne blinde Flecken“: nicht behaupten; Architektur muss anhand Bedrohungsmodell, implementierter Tests, Betriebserfahrung und Compliance-Review validiert werden.

17. Quellen und Nachweise

Die technischen Verweise hier dienen der Verifikation zentraler Entscheidungen. Für den vollständigen Projekt-Handoff die verlinkten Dokumente in ADRs/Repo dokumentieren und genutzte Bibliotheksversionen pinnen.

• LangGraph Interrupts: persistent checkpointer/thread_id, resume semantics, idempotente Seiteneffekte, typed response schema. https://docs.langchain.com/oss/python/langgraph/interrupts

• PostgreSQL Row Security: policies, default-deny, table-owner/BYPASSRLS caveats, FORCE RLS, backup caution. https://www.postgresql.org/docs/current/ddl-rowsecurity.html

• IETF RFC 9700 OAuth 2.0 Security BCP: Password Grant MUST NOT be used; refresh-token protections, PKCE guidance. https://www.rfc-editor.org/rfc/rfc9700.html

• arc42 Section 8: Cross-cutting concepts. https://docs.arc42.org/section-8/

• Ursprungsdokumente: „NexusSaaS: KI-Recherche-Auftrag & JSON-Blueprint“ und „Validierung und architektonische Erweiterung der NexusSaaS-Spezifikation für den Entwickler-Handoff“; beide wurden als Ausgangsmaterial konsolidiert.

Abgrenzung: Diese PDF ist ein Architektur-/Umsetzungsentwurf. Vor Produktionsbetrieb müssen Quellcode, Cloud-Konfiguration, Bedrohungsmodell, Datenverarbeitungsverträge, Datenschutzinformation, Lösch-/Restore-Prozesse, Penetrationstests und Betriebsziele separat geprüft und freigegeben werden.